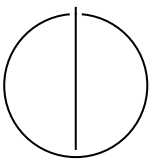
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Reduction of Memory in Tensor Network Contractions

Fam Shihata



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

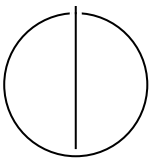
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Reduction of Memory in Tensor Network Contractions

Speicherreduzierung bei Tensornetzwerkkontraktionen

Author:	Fam Shihata
Examiner:	Dr. Mohammed Shahzad
Supervisor:	Manuel Geiger
Submission Date:	Submission date



I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, Submission date Fam Shihata

Acknowledgments

Abstract

Contents

1 Introduction

Tensor network contraction (TNC) has become a core computational primitive in fields ranging from quantum many-body physics and quantum circuit simulation to chemistry and high-dimensional machine learning and data analysis [8]. Their appeal comes from the ability to represent and manipulate exponentially large objects through structured factorizations, often making previously intractable problems accessible in practice [6]. Yet this promise comes with a major obstacle: the cost of contracting a tensor network can vary significantly depending on how intermediate tensors are formed. In many realistic settings, the limiting factor is not only the number of arithmetic operations, but the peak memory required to store these intermediates. A contraction strategy that is theoretically efficient in floating-point operations (FLOP) may still be unusable in practice if it creates tensors too large to fit in memory.

This makes memory reduction a central problem rather than a secondary implementation concern. Prior work on contraction ordering has already shown that the sequence in which tensors are contracted has a decisive impact on computational feasibility and overall performance [20, 21]. At the same time, high-performance tensor contraction frameworks and tensor-transposition libraries have demonstrated that practical efficiency depends heavily on how contractions are mapped onto hardware, how data is rearranged, and how temporary storage is managed [13, 24]. In other words, optimizing tensor networks requires attention not only to algebraic structure, but also to memory footprint, data movement, and layout-aware execution.

The importance of optimizing tensor network contractions originates from both theoretical and practical considerations. From a theoretical perspective, tensor contraction is known to be computationally hard in general, with complexity that can grow exponentially with the size of the network [20]. From a practical standpoint, modern applications such as quantum circuit simulation and many-body physics rely heavily on large-scale tensor contractions, where inefficiencies quickly become restrictive.

A key observation in high-performance computing (HPC) is that memory bandwidth and data movement increasingly dominate execution time, often surpassing the cost of FLOPs [13, 14]. In large-scale simulations, such as those targeting quantum advantage benchmarks [14], the ability to efficiently manage memory and minimize data movement is critical to achieving feasible runtimes.

Moreover, the peak memory usage during tensor contraction directly determines

whether a computation is even possible on a given system. Many tensor network contractions fail not due to computational limits, but because intermediate tensors exceed available memory [11]. As a result, reducing peak memory usage is not merely an optimization objective, but a fundamental requirement for scalability.

Despite these challenges, most existing optimization techniques primarily focus on reducing computational cost (FLOPs) through contraction ordering, often overlooking memory-related factors such as tensor layout, data locality, and permutation overhead [13, 21]. This creates a gap between theoretical optimization and practical performance, particularly on modern architectures where memory hierarchy plays a central role.

1.1 Problem Formulation and Challenges

Ultimately, the goal of contraction is to compute a final tensor by successively contracting pairs of tensors along shared indices. This process involves three key components: the contraction order, which determines the sequence in which tensor pairs are combined; the intermediate tensor shapes, which define the sizes and dimensionalities of tensors generated during the contraction; and the tensor layout and permutation, which determine how tensor indices are arranged in memory and therefore how efficiently operations can be executed.

While contraction ordering has been extensively studied, the challenges associated with intermediate tensor shapes and tensor layouts remain significant and often underexplored. In particular, tensor permutations play a crucial role in enabling efficient computation. High-performance implementations typically rely on transforming tensor contractions into matrix multiplications via reshaping and permutation, a technique known as Tensor-Tensor Generalized Multiplication (TTGT) [24].

However, these permutations introduce additional memory movement, which can be costly. In many cases, tensors are repeatedly permuted and reshaped throughout the contraction sequence. This leads to increased memory bandwidth usage, poor cache locality, and higher peak memory consumption due to the creation of intermediate copies. These effects collectively degrade performance and can become a dominant bottleneck in large-scale computations.

Furthermore, existing approaches typically treat permutations as local operations required to enable individual contractions. This local perspective ignores the global structure of the contraction sequence and can result in redundant or suboptimal data transformations that accumulate over the course of execution [4].

1.2 Research Aim

This thesis proposes a complementary approach that shifts part of the optimization focus to post-contraction path discovery. Instead of solely optimizing the contraction order, we investigate how global tensor permutations can be applied after determining a contraction path to improve memory efficiency.

The key idea is to treat tensor layout as a first-class optimization variable. By analyzing the entire contraction sequence, it becomes possible to reduce redundant permutations across consecutive contractions, improve data locality and cache utilization, and minimize peak memory usage by carefully controlling intermediate tensor layouts.

This approach is relatively niche compared to traditional methods, as most existing work focuses on pre-contraction optimization and local permutation strategies. However, given the growing importance of memory efficiency in modern computing systems, global permutation optimization presents a promising direction for further research.

1.3 Contributions of This Work

In summary, this thesis makes several key contributions. It introduces a novel framework for post-contraction optimization based on global tensor permutations. It provides a detailed analysis of the impact of tensor layouts on memory usage and data movement. It highlights the limitations of existing pre-contraction optimization techniques in addressing memory-related challenges. Finally, it demonstrates how tensor permutation strategies can complement contraction ordering to achieve improved overall performance. By bridging the gap between contraction planning and memory-aware tensor layout optimization, this work aims to provide a more comprehensive approach to efficient tensor network contraction.

2 Background

2.1 Foundations of Tensor Networks

2.1.1 Tensors

A tensor is a multidimensional array that generalizes the concept of vectors and matrices to arbitrary dimensions. Formally, a tensor of order n (n-mode tensor or rank n tensor) is a collection of entries indexed by n indices, each ranging over a finite set of dimension sizes. For instance, a matrix is a second-order ($n = 2$) tensor with two indices, while a vector is a first-order ($n = 1$) tensor with a single index (see Figure 2.1). Tensors are fundamental objects in linear algebra and multilinear computations, enabling compact representations of high-dimensional data and operations on them. The elements of a tensor $\mathcal{T}$ are accessed via multi-indices, denoted $\mathcal{T}_{i_1,i_2,\ldots,i_n}$, where each i_k ranges from 1 to d_k, the size of dimension k. The shape of a tensor, given by the tuple $(d_1, d_2, \ldots, d_n)$, determines the total number of entries, which scales as the product of all dimension sizes. This exponential growth in storage and computation with dimension and order motivates the use of structured factorizations and tensor networks, which exploit sparsity and algebraic structure to make high-dimensional problems computationally tractable.

2.1.2 Tensor Contractions

Tensor contractions have emerged as one of the fundamental operations on tensors. At its core, a tensor contraction represents a generalized summation over shared indices between tensors. This operation constitutes many familiar linear algebra operations such as matrix multiplication, but extends naturally to higher-order interactions. Formally, a tensor contraction between two tensors $\mathcal{A}$ and $\mathcal{B}$ over a set of shared indices $S = \{i_1, i_2, \ldots, i_r\}$ is defined as:

$$\mathcal{C}_{j_1,\ldots,j_m,k_1,\ldots,k_n} = \sum_{i_1=1}^{d_1} \sum_{i_2=1}^{d_2} \cdots \sum_{i_r=1}^{d_r} \mathcal{A}_{i_1,\ldots,i_r,j_1,\ldots,j_m} \cdot \mathcal{B}_{i_1,\ldots,i_r,k_1,\ldots,k_n} \tag{2.1}$$

where indices $i_1, \ldots, i_r$ are the contracted (summed) indices shared between $\mathcal{A}$ and $\mathcal{B}$, and indices $j_1, \ldots, j_m$ and $k_1, \ldots, k_n$ are the free indices that remain in the result.

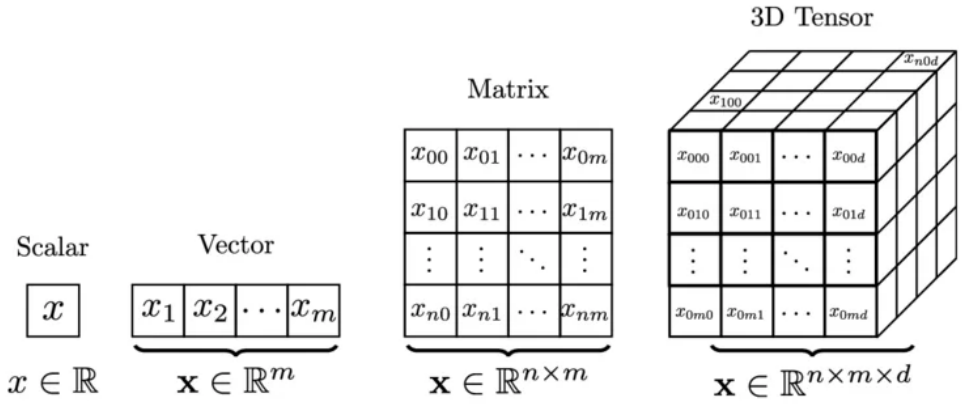


Figure 2.1: Examples of rank-0, 1, 2, 3 tensors (i.e. a scalar, vector, matrix, order-3 tensor)

A more compact and standard representation of tensor contractions is given by Einstein notation (Einstein summation convention), where repeated indices are implicitly summed over. For instance, the contraction above can be written concisely as:

$$\mathcal{C}_{jk} = \mathcal{A}_{ij}\mathcal{B}_{ik} \tag{2.2}$$

where i is the repeated (contracted) index and is implicitly summed over all its dimensions, while j and k are free indices appearing in the result. This notation extends naturally to higher-order tensors and multiple contractions, making it a powerful tool for expressing complex tensor network computations in a compact form.

2.1.3 Tensor Networks

However, this is not the only way to effectively represent complex contractions of networks; we can also draw them in tensor networks. A tensor network is a graphical representation of a set of tensors and their contraction structure, where nodes represent tensors and edges represent shared indices.

In this graphical framework, each tensor is depicted as a node (typically drawn as a circle, square, or other shape), with a number of lines (legs) extending from it equal to the tensor's order. Each leg corresponds to one index of the tensor. When two legs are connected by an edge, the corresponding indices are contracted (summed over). Free indices, which do not participate in contractions, are drawn as dangling legs extending from the network.

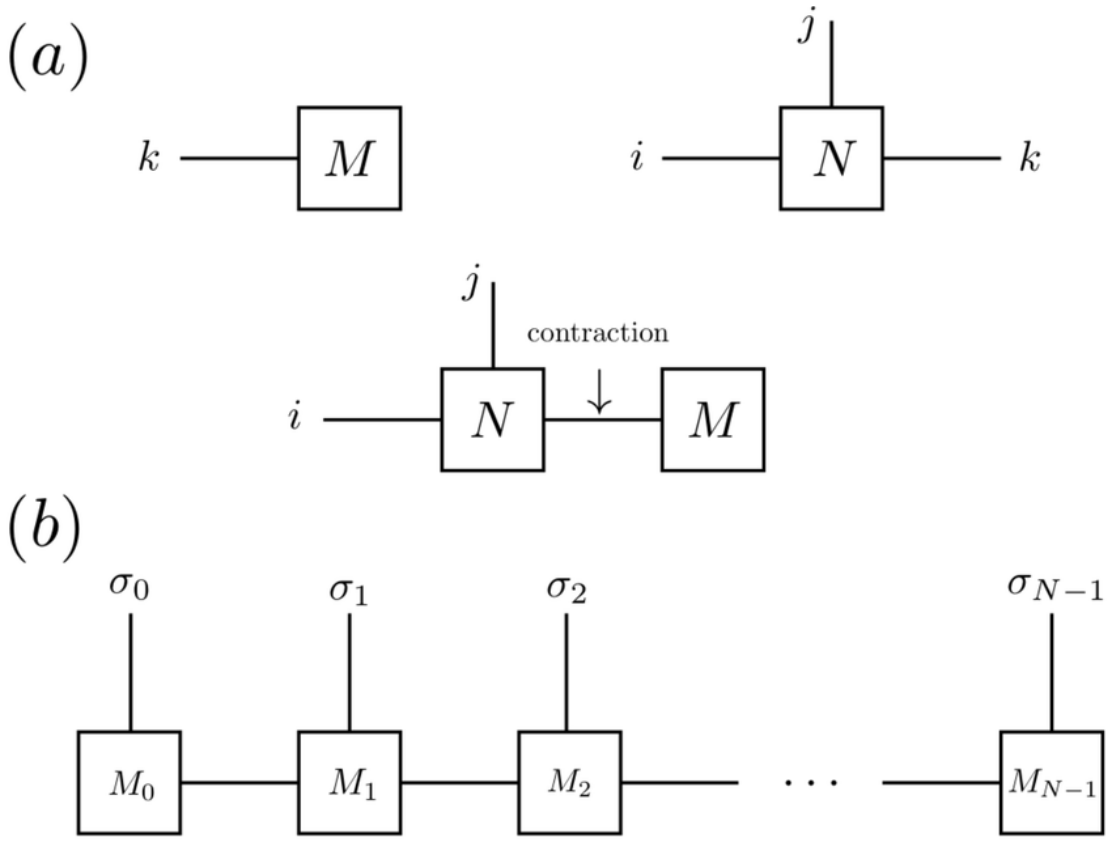


Figure 2.2: (a), on the top, displays a vector, M, and a rank-3 tensor, N, contracted across their common leg k; (b), on the bottom, displays a complex network of N tensors chained in a tensor train, commonly known as a Matrix Product State (MPS)

Consider a simple example: contracting two tensors $\mathcal{A}_{ij}$ and $\mathcal{B}_{jk}$ to produce $\mathcal{C}_{ik}$. In tensor network notation, $\mathcal{A}$ appears as a node with two legs (for indices i and j), and $\mathcal{B}$ appears as a separate node with two legs (for indices j and k). The leg corresponding to index j in $\mathcal{A}$ is connected to the leg corresponding to index j in $\mathcal{B}$, indicating that these indices are contracted. The remaining legs (corresponding to i and k) dangle freely, representing the free indices of the result tensor $\mathcal{C}$. Figure 2.2 illustrates this concept with more examples.

More complex tensor networks can compactly represent intricate contraction patterns that would be cumbersome to express algebraically. The graphical representation immediately reveals the structure of the network i.e., which tensors share indices,

the connectivity pattern, and which indices remain uncontracted. This makes tensor network diagrams important for visualizing and reasoning about large-scale contractions [2].

2.2 Basic Linear Algebra Subprograms

Basic Linear Algebra Subprograms (BLAS) is a standardized specification for low-level linear algebra operations that serves as a foundation for high-performance numerical computing. Originally developed in the late 1970s [15], BLAS provides a portable and efficient interface for fundamental vector and matrix operations, enabling libraries and applications to achieve near-optimal performance across diverse hardware architectures.

BLAS operations are organized into three hierarchical levels, each characterized by the computational intensity (ratio of arithmetic operations to memory accesses) and scope of the operations:

2.2.1 Level 1: Vector Operations

Level 1 BLAS conserns with scalar and vector operations with computational complexity $O(n)$, where n is the vector length. These operations include vector addition, scalar multiplication, dot products, and norm calculations. Due to their linear computational intensity and high memory-to-computation ratio, Level 1 operations are memory-bound and exhibit poor cache utilization. Examples include `AXPY` (scaled vector addition) (2.3), `DOT` (dot product), and `NRM2` (Euclidean norm).

$$y \leftarrow \alpha x + y \tag{2.3}$$

2.2.2 Level 2: Matrix-Vector Operations

Level 2 BLAS optimizes matrix-vector operations with computational complexity $O(n^2)$, including matrix-vector products, rank-one updates, and triangular system solves. These operations have moderate computational intensity and remain memory-bound on most modern architectures. Common Level 2 operations include `GEMV` (general matrix-vector multiply) (2.4) and `GER` (general rank-one update).

$$y \leftarrow \alpha A x + \beta y \tag{2.4}$$

2.2.3 Level 3: Matrix-Matrix Operations

Level 3 BLAS covers matrix-matrix operations with computational complexity $O(n^3)$, such as matrix multiplication and triangular solves. These operations exhibit high computational intensity and can achieve efficient cache reuse through careful data blocking and tiling strategies. Level 3 operations are typically compute-bound on modern hardware, making them suitable targets for optimization and parallelization. The general matrix-matrix multiplication (GEMM) (2.5) operation is the cornerstone of Level 3 BLAS.

$$C \leftarrow \alpha AB + \beta C \tag{2.5}$$

Efficient implementations of BLAS, such as ATLAS, OpenBLAS, and Intel MKL, are critical for achieving high performance in scientific computing and machine learning applications [10]. In the context of tensor contractions, BLAS operations, particularly Level 3 GEMM, serve as the primary computational kernel when contractions are transformed into matrix multiplications.

2.3 High-performance Tensor Contraction Implementations

The work presented uses the `Tensor Contraction Code Generator (TCCG)` [23] library as its high-performance tensor contraction engine. `TCCG` is designed specifically to deliver efficient GEMM kernels across modern CPU architectures by combining the algorithmic principles of blocked matrix multiplication with architecture-aware microkernel generation.

At the highest level, `TCCG` decomposes the computation into a hierarchy of tiles. The input matrices are partitioned into blocks that fit into the different levels of the processor cache, which reduces main-memory traffic and improves data reuse. The blocked structure also enables the library to expose parallelism naturally: independent blocks can be computed concurrently across CPU cores, while each core operates on cache-resident panels of the matrices.

A central feature of `TCCG` is its use of a carefully tuned microkernel. The microkernel computes a small block of the result matrix using registers and vector instructions, often with explicit loop unrolling and packing of input matrix panels. This organization minimizes the number of memory accesses per floating-point operation, which is essential for reaching high fractions of peak performance on modern processors.

The library also supports explicit matrix packing, where subblocks of the input matrices are copied into contiguous, cache-friendly buffers before the inner product is evaluated. Packing eliminates irregular memory access patterns that occur in standard

row-major or column-major layouts, and it allows the microkernel to consume data in a streaming fashion. In TCCG, the packed panels are chosen to align with the target architecture's cache line size and vector width.

Beyond blocking and packing, TCCG implements an adaptive blocking strategy for common matrix shapes. For square matrices, the blocking factors are selected to balance between the L1 / L2 cache capacities and the cost of data movement. For rectangular matrices, the library selects block sizes to maintain high utilization of the inner microkernel while avoiding excessive padding or wasted computation.

It is important to emphasize that TCCG is not just treated as a black-box library, but integrated into the overall implementation. Its performance model is based on the same principles discussed earlier in this chapter: increasing arithmetic intensity, maximizing cache reuse, and exposing parallelism through block-level decomposition. When tensor contractions are mapped to matrix-matrix multiplications, TCCG provides the low-level kernel that performs these multiplications efficiently, making the overall tensor contraction implementation practical for large-scale problems.

2.4 High-performance Tensor Transposition Implementations

The main tensor transposition engine used in this work is the High-Performance Tensor Transposition (HPTT) library [25]. HPTT addresses a data-movement-centric bottleneck that is ubiquitous in tensor-based codes. While a single GEMM call can be made very fast, the full contraction pipeline often requires rearranging tensor data, and inefficient transpositions can dominate runtime if not handled carefully.

HPTT implements multidimensional, out-of-place tensor transpositions using the same high-performance principles introduced earlier for matrix multiplication: blocking, explicit vectorization, a small hand-tuned micro-kernel, and parallelization. Concretely, HPTT decomposes any higher-order permutation into a nest of loops surrounding a micro-kernel that performs a small, cache-resident element reordering. This decomposition has three practical advantages: (1) the micro-kernel can be hand-vectorized for the target instruction set (e.g., AVX, NEON), (2) blocking at multiple levels improves cache reuse and reduces off-chip bandwidth requirements, and (3) the loop nest exposes both data- and task-level parallelism for multicore execution.

$$B_{\Pi(i_1,i_2,\ldots,i_N)} \leftarrow \alpha \times A_{i_1,i_2,\ldots,i_N} + \beta \times B_{\Pi(i_1,i_2,\ldots,i_N)}$$

Figure 2.3: Illustration of HPTT's tensor transposition semantics and permutation notation (image source: springer13/hptt).

An important engineering feature of HPTT is its optional autotuning framework. At

plan creation time, the library can either estimate good implementation parameters from a performance model (fast) or explore a search space of loop orders, blocking factors, and parallelization strategies (measure/patient modes) to find the best-performing schedule for the current tensor shape and hardware. This behavior is analogous to FFTW's planner [7] and makes HPTT robust in applications where tensor sizes and permutations are only known at runtime.

HPTT also supports explicit matrix/tensor packing and specializes micro-kernels to align with cache line sizes and vector register widths, thereby minimizing irregular memory accesses and enabling streaming consumption of input data. The library exposes simple C, C++, and Python interfaces for creating a transposition plan (with or without autotuning) and executing it, for example see Figure 2.4.

```
#include <hptt.h>

// allocate tensors
float A* = ...
float B* = ...

// specify permutation and size
int dim = 6;
int perm[dim] = {5,2,0,4,1,3};
int size[dim] = {48,28,48,28,28};

// create a plan (shared_ptr)
auto plan = hptt::create_plan( perm, dim,
                           alpha, A, size, NULL,
                           beta, B, NULL,
                           hptt::ESTIMATE, numThreads);

// execute the transposition
plan->execute();
```

Figure 2.4: Example usage of HPTT's C++ interface from the library's documentation.

In practice, integrating HPTT as the transposition layer in a tensor contraction pipeline significantly reduces the overall runtime, because the cost of rearranging tensor data becomes comparable, if not superior, to the cost of the high-performance GEMM kernels if not optimized.

3 Related Work

Early work primarily optimized for FLOPs, but it is now well understood that minimizing FLOPs alone can lead to prohibitively large intermediate tensors and thus excessive memory consumption. Therefore, a new set of algorithms have been developed to reduce this effect of the memory. This section reviews these contributions, with a focus on techniques that explicitly reduce memory footprint or data movement.

3.1 Treewidth-Based and Graph-Theoretic Methods

Markov and Shi [17] established a foundational connection between tensor network contraction and graph theory, showing that the complexity of contraction is governed by the treewidth of the underlying graph. Minimizing treewidth implicitly controls the size of intermediate tensors and therefore peak memory usage. This is due to the treewidth bounding the size of the largest intermediate tensor produced during contraction. A contraction with treewidth k guarantees that no intermediate tensor will have more than k indices. This directly translates to memory savings. Moreover, low treewidth enables efficient dynamic programming algorithms that can compute the optimal contraction sequence in time exponential only in treewidth, not in the total number of tensors.

However, exact treewidth minimization is NP-hard [5], motivating heuristic approaches. Subsequent work has leveraged graph partitioning and elimination ordering heuristics to approximate optimal contraction paths. For example, tools such as `quickbb` [9] and `flow-cutter` [12] have been adapted to tensor contraction problems. These approaches reduce peak memory by limiting separator sizes, though they are not explicitly designed for memory movement.

3.2 Hypergraph Partitioning Approaches

More recent work models tensor networks as hypergraphs, enabling more accurate representation of multi-index tensors. Gray and Kourtis [11] introduced a hypergraph-based contraction path optimizer that uses partitioning techniques to minimize both

FLOPs and intermediate tensor sizes. This framework allows explicit incorporation of memory constraints and has been widely adopted in tools such as `cotengra`.

Hypergraph partitioning methods can be extended to multi-objective optimization, balancing FLOPs and memory. For instance, multi-criteria cost functions penalize large intermediate tensors, effectively reducing peak memory usage. However, these methods typically focus on static path selection and do not account for runtime memory movement.

3.3 Slicing Techniques

Slicing is one of the most widely used methods for reducing peak memory. It involves fixing certain indices to specific values, thereby decomposing a large contraction into multiple smaller subproblems. Each slice can be computed independently, significantly reducing memory requirements at the cost of increased total computation. Figure 3.1 illustrates this process.

This approach has been extensively used in quantum circuit simulation, including in Google's quantum supremacy experiments [1]. Later work formalized slicing as an optimization problem, selecting indices to slice in order to trade off memory and runtime [11]. In practice, slicing is especially attractive when the memory footprint of the original contraction exceeds available RAM, because it can reduce the size of intermediates by several orders of magnitude.

However, slicing also introduces a trade-off between memory and execution cost that must be carefully managed. In the context of quantum circuit simulation, Google's quantum supremacy experiment used slicing to reduce the peak memory of a 53-qubit simulation, at the expense of performing many more independent contractions than in the unsliced case [1].

The benefits of slicing are not limited to raw memory reduction. Because individual slices are independent, they expose a high degree of parallelism and can be distributed across processors or compute nodes with minimal synchronization. This property is especially advantageous for out-of-core or distributed execution, where fitting a single slice into local memory or cache can substantially improve throughput. Several works therefore treat slicing as a practical mechanism to scale tensor contraction to larger circuits or networks than would otherwise fit on a single device.

While slicing is a powerful technique for enabling contractions that would otherwise be infeasible due to memory limits, it is most effective when used in combination with path optimization and execution-level strategies. Careful selection of slicing indices and integration with low-level data movement optimization are necessary to realize the potential benefits without incurring prohibitive runtime overhead.

3.4 Recompression and Tensor Decomposition

Another class of methods reduces memory by compressing intermediate tensors. Techniques such as singular value decomposition (SVD) and tensor train (TT) decompositions are used to approximate tensors with lower-rank representations.

For example, methods based on matrix product states (MPS) [22] and tensor trains [18] dynamically truncate intermediate tensors to control memory growth. These approaches are particularly effective in structured problems with low entanglement, but introduce approximation error and may not generalize to arbitrary tensor networks which are the primary focus of this work.

3.5 Memory Movement and Data Locality

While peak memory has received significant attention, memory movement is increasingly recognized as a critical bottleneck. Modern hardware architectures exhibit large disparities between computation speed and memory bandwidth, making data movement a dominant cost.

Efforts to reduce memory movement include blocking and tiling strategies [16, 19], which aim to maximize cache reuse, as well as layout transformations that improve data locality. In the context of tensor contractions, permutation of tensor indices plays a key role in determining memory access patterns.

However, most existing work treats permutations as a secondary concern, focusing primarily on algebraic correctness rather than memory efficiency. This represents a significant gap in the literature.

3.6 Summary and Research Gap

The literature on tensor network contraction reveals two complementary lines of work. One line emphasizes pre-contraction optimization, using graph-theoretic and hypergraph partitioning techniques to choose contraction orders that limit intermediate tensor sizes and reduce peak memory across the execution. The other line emphasizes post-contraction execution, using slicing, recompression, and out-of-core scheduling to cope with cases where available memory is insufficient for a single monolithic contraction. Both approaches have advanced the ability to solve larger networks and higher-dimensional problems, but they also expose an important limitation: the predominant focus has been on peak memory and arithmetic cost, while the costs of data movement, tensor layout, and permutation overhead are still treated as subordinate concerns.

This gap is especially salient in modern high-performance contexts, where memory bandwidth and cache locality often dominate actual runtime. As the introduction of this thesis highlights, a contraction strategy with low theoretical FLOP count can still be impractical if it produces intermediates whose layout requires repeated reshaping and transposition. Existing work on contraction ordering and path selection therefore needs to be complemented by optimization that accounts for the global structure of tensor permutations and their impact on memory movement.

Accordingly, the contribution of this thesis is to bridge the divide between contraction planning and execution-level memory optimization. By treating tensor layout as a first-class optimization variable and by analyzing post-contraction permutations across an entire contraction sequence, this work aims to reduce redundant data transformations, improve locality, and achieve better memory efficiency without sacrificing the benefits of established path optimization techniques. In doing so, it seeks to provide a more holistic foundation for efficient tensor network contraction on modern architectures.

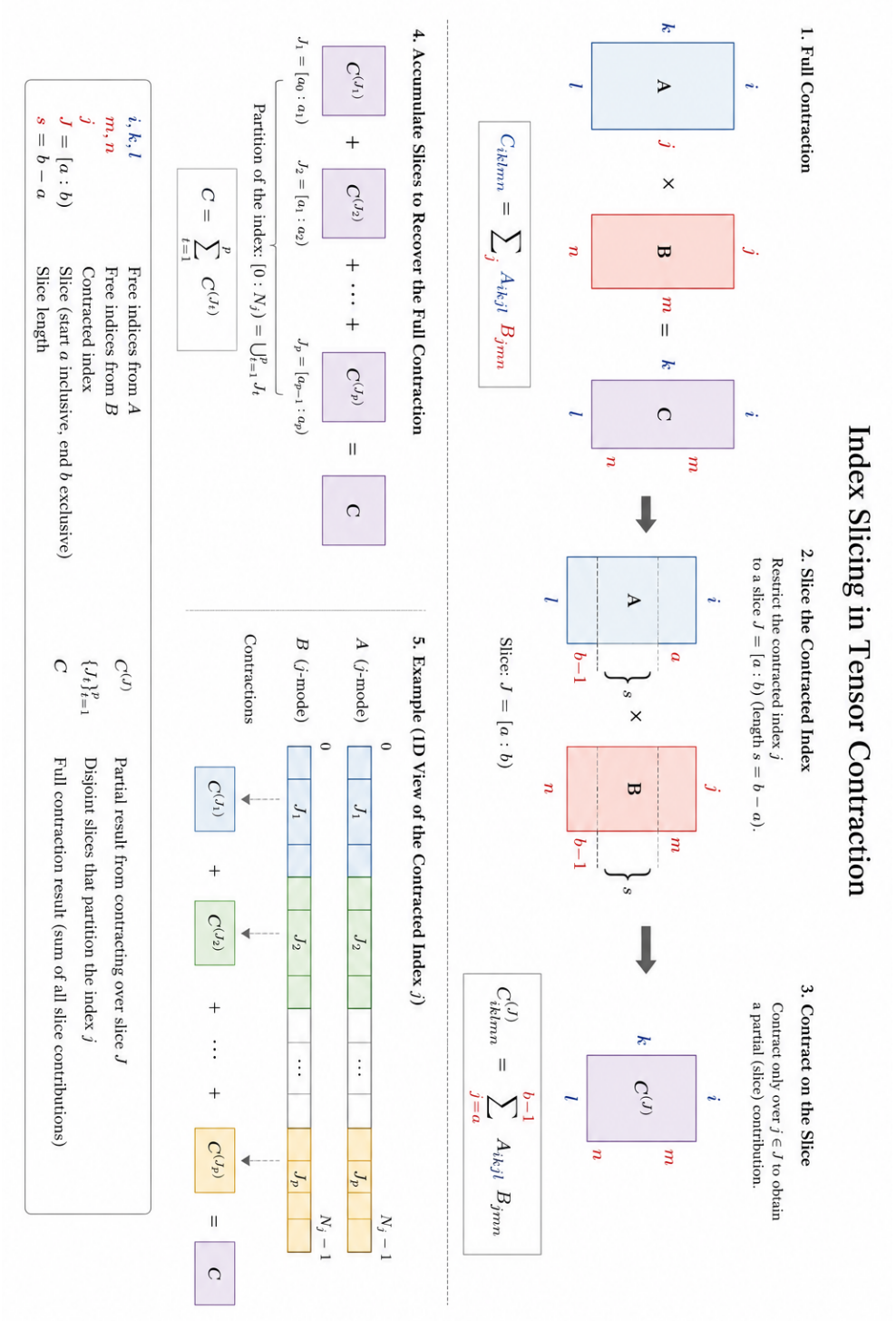


Figure 3.1: Illustration of index slicing in tensor contraction

4 Implementation

The purpose of this chapter is to outline the implementational details of the library developed, the tensor permutation discovery algorithm alongside the required datastructues, and the end-to-end pipeline used for compiling the operations. We provide the reader with a top-down description of our approach, stepping down one abstraction at a time, i.e. from the library's interface to the core of our algorithm to how it is integrated into a state-of-the-art pipeline.

4.1 Interface

The library exposes a concise Python interface, see Figure 4.1, for permutation strategies that is independent of the concrete optimization algorithm. This interface is defined as a runtime-checkable protocol and makes it possible to register and interchange different strategy implementations without changing the surrounding compilation pipeline.

```
@runtime_checkable
class IStrategy(Protocol):
    @staticmethod
    def find_optimal_permutation(
        network: TensorNetwork, contraction_path: ContractionPath
    ) -> list[TensorOperation]:
        ...
```

Figure 4.1: Outgoing python interface for permutation discovery strategies.

This interface is the library's public contract for strategy implementations. The `find_optimal_permutation` method is responsible for producing an ordered sequence of tensor operations that realize an optimized contraction plan.

The strategy implementation receives two explicit inputs: a tensor network and a contraction path. The tensor network is represented by the `TensorNetwork` dataclass, see Figure 4.2, which contains the initial tensor collection, the desired output indices, and a size map. The contraction path is a sequence of index pairs that specifies

the pairwise contraction order. Internally, the tensor network manages its tensors through a private `_Tensorpool` abstraction, enabling dynamic reuse of tensor objects across contraction steps instead of allocating repeated tensor storage. This decision is important for performance, and it will be highlighted again in later sections when discussing the algorithm's datastructures and execution behavior. Finally, a strategy's return type is not a raw permutation matrix or a list of index orders, but an explicit intermediate representation in the form of `TensorOperation` objects.

```
@dataclass(init=False)
class TensorNetwork:
    tensors: _TensorPool
    output_indices: list[int]
    size_dict: dict[int, int]
```

Figure 4.2: Snippet of the tensor network dataclass.

The intermediate representation, see Figure 4.3, decouples permutation planning from execution. It allows the optimizer to express both layout transformations and contraction steps as first-class operations that can be later compiled, scheduled, or simulated by the pipeline.

```
class TensorOperation(ABC):
    @abstractmethod
    def apply(
        self, *inputs: TensorOperationResult, **kwargs: dict[str, Any]
    ) -> TensorOperationResult:

class TensorPermutationOperation(TensorOperation):
    ...

class TensorContractionOperation(TensorOperation):
    ...
```

Figure 4.3: Snippets of the intermeidate representation objects.

Additionally, strategies expose a `get_peak_memory` method for a second aspect of the interface: cost estimation. It computes the maximum memory footprint that a given path and its permutations will require during execution. Similarly, `get_total_memory` reports the total memory movement associated with the same plan. These metrics are

useful for comparing candidate strategies and selecting one that balances performance and resource usage.

By defining both permutation planning and memory estimation in the same interface, the library provides a coherent abstraction for strategy selection. Specific algorithms, such as the greedy strategy developed in this work, implement this protocol and are therefore interchangeable with alternative strategies that may be added in the future.

4.2 Preparation and Peripherals

4.2.1 Persistent Contraction Path

Before the permutation strategy can be applied, the contraction path must be prepared in a form that preserves the complete tensor network history. The library uses a dedicated `PersistentContractionPath` abstraction for this purpose. This class stores the original contraction path together with a history of tensor network states, and enforces that the history contains exactly one entry for the initial state plus one additional entry for each contraction step.

The persistent path is inspired by persistent data structures such as segment trees, where later versions share unchanged portions of earlier versions through structural sharing. In this implementation, the contraction history is built by simulating each contraction in order and appending only the changed state to the history list while preserving references to unchanged tensors wherever possible. This means that the algorithm can jump directly to any intermediate step without replaying the entire contraction sequence, and that the storage cost grows only with the number of actual changes rather than with a naive full copy of every intermediate network.

The `from_contraction_path` factory method is the entry point for preparing a contraction path. It takes the original tensor network and the proposed path, then constructs the sequence of states needed by the permutation strategy. The history is kept immutable from the strategy's perspective through the use of deep copies when states are retrieved, ensuring that later operations cannot accidentally mutate earlier steps. As a result, both correctness and reproducibility are preserved when the optimizer analyzes different nodes in the contraction tree.

This persistent representation is therefore not only a convenience feature, but a core preparatory step for the algorithm. It is essential for the strategy's ability to inspect the tensor network at arbitrary points in the contraction sequence without recomputing the entire tree, and it ensures that the memory footprint of the preparation phase remains proportional to the number of contracted states rather than to the size of each complete tensor network copy.

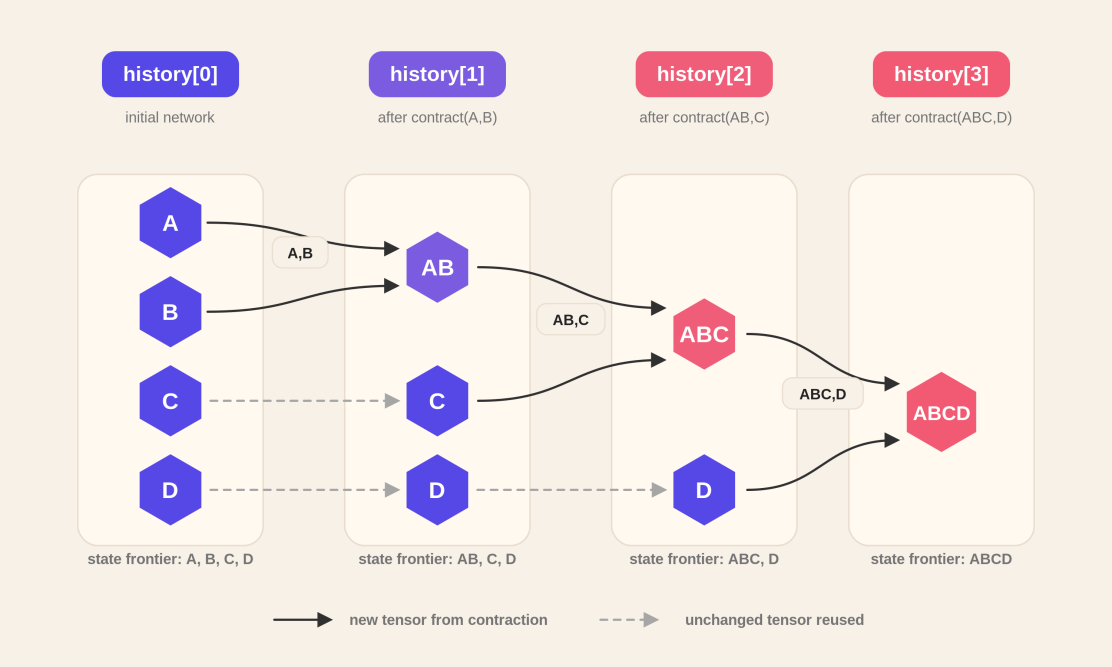


Figure 4.4: Illustration of the persistent contraction path and state history.

4.2.2 APIs and ABIs

The library exposes a dedicated interface module under `tccg_interface`, which acts as the bridge between the tensor contraction logic and the low-level TCCG and runtime. At the highest level, `execute_tccg_contraction` orchestrates the generation of a TCCG description for a given pair of input tensors, discovers the generated function signature, compiles a `pybind11` wrapper, and finally loads and executes the resulting shared object.

This Python-side API hides the details of the TCCG DSL, the C++ code generation pipeline, and the shared library loading semantics behind a single callable contract. The `tccg_interface` package is implemented as a layered ABI stack, with `TCCGGenerator` responsible for producing the `.tccg` description and invoking the installed `tccg` executable, `TCCGDiscoverer` parsing the resulting generated source, `TCCGPyBind11Compiler` compiling the wrapper into a native module, and `TCCGRuntime` loading the compiled module dynamically.

Because the runtime module is created through `pybind11`, the library's ABI is defined by the exact function signature produced by TCCG and exposed through the generated wrapper. The compilation step therefore requires access to the native HPTT headers

and libraries, which are discovered from the environment variables `HPTT_ROOT` and `BLIS_ROOT`. The generated TCCG kernels are linked against HPTT to provide the high-performance transpose and packing primitives that are necessary for the matrix-matrix multiplication kernels to achieve practical throughput.

The compiler glue is implemented in `TCCGPyBind11Compiler`, which emits a small C++ wrapper around the generated TCCG function and then invokes the resepctive C++ compiler with the appropriate include and link paths.

This explicit ABI binding ensures that the tensor contraction kernels generated by TCCG are directly compatible with the HPTT-backed transposition and workspace management used elsewhere in the library. Thus, the `tccg_interface` package serves as the central integration point between the algorithmic layer in Python and the native tensor compute stack defined by TCCG.

4.3 The Greedy Algorithm

The greedy permutation strategy is the core algorithmic contribution of this work. Rather than exhaustively exploring all possible permutation combinations, which is computationally expensive, the strategy employs a greedy prioritization heuristic that focuses computational effort on the tensors where permutation decisions have the greatest impact on overall execution cost.

The fundamental insight underlying the greedy approach is that not all tensor permutations are equally consequential. Within a given contraction tree, the tensors with the largest memory footprints are the ones whose layout decisions dominate the total memory movement cost. By identifying these "peak" tensors and computing GEMM-friendly layouts for them without permutation overhead, the strategy can then propagate compatible layouts backward through the contraction tree, ensuring that the peak tensors remain unpermuted while all other tensors receive permutation operations as needed to maintain compatibility.

This section provides a comprehensive description of the algorithm, decomposing it into logical components: the peak node identification phase, the target layout computation phase, the recursive planning phase, and the permutation assignment phase.

4.3.1 Overview and Initialization

In addition to the default inputs from the interface, the method accepts a third parameter an integer parameter k, allowing users to tune the strategy. For example, $k = 1$ focuses all permutation effort on the single largest tensor, the "peak", while larger values of k

distribute the permutation budget across more tensors, potentially reducing contraction-time memory pressure but increasing total permutation overhead.

The method then prepares a `PersistentContractionPath`. This preparation is essential because the algorithm requires random-access queries to intermediate network states to reason about which tensors are the largest and to determine their positions and index structures. By constructing the persistent path upfront, subsequent queries avoid the expensive recomputation of contraction steps.

```
def FIND_OPTIMAL_PERMUTATION(network, contraction_path, k):
    validate_k(k, network, contraction_path)
    persistent_path = build_persistent_contraction_path(network,
        contraction_path)
    initial_perms = identity_permutations(network.tensors)
    intermediate_perms = identity_permutations(
        persistent_path.intermediate_results)

    if persistent_path.num_steps == 0:
        return materialize_operations(initial_perms,
            intermediate_perms, contraction_path)

    tree, leaf_to_node, step_to_node = build_tree_maps(persistent_path)
    peak_nodes = find_peak_nodes(network,
        persistent_path, leaf_to_node, step_to_node, k)

    desired_layouts = {}
    for node in peak_nodes:
        desired_layouts[node.id] = compute_peak_target_layout(node,
            desired_layouts)

    plan_recursive_layouts(tree.root, desired_layouts,
        frozen_nodes=peak_nodes)
    return materialize_operations(initial_perms, intermediate_perms,
        contraction_path)
```

Figure 4.5: Pseudocode for the overview and initialization phase of the greedy strategy.

4.3.2 Finding Peak Nodes

The first major algorithmic phase identifies which tensors in the contraction tree should be treated as "peaks". In this step, a unified list of candidate tensors that includes both initial tensors from the network and intermediate results produced during the contraction steps are constructed. For each candidate, it computes an estimate memory cost.

Once all candidates are gathered and assigned memory costs, the function sorts them in descending order of memory. It then performs a deduplication pass to select the top k unique tensors by identity. Because the same contraction step might be referenced multiple times in the candidate lists, deduplication ensures that each tensor is counted only once. Finally, the selected nodes are sorted by their leaf status (leaf nodes first), returning them in a stable order that respects both the memory priority and structural properties. It is important to note that failing to freeze leaf nodes first might lead to an infeasible problem. The corresponding selection logic is illustrated in Figure 4.6.

```
def FIND_PEAK_NODES(network, persistent_path,
    leaf_to_node, step_to_node, k):
        candidates = []
        candidates += largest_tensors(network)
        candidates += largest_intermediate_tensors(persistent_path)

        sort_descending(candidates, key=memory_size)
        unique_candidates = deduplicate_by_identity(candidates)
        selected = unique_candidates.take_first(k)

        return sort_by_leaf_status(selected)
```

Figure 4.6: Pseudocode for selecting the top-k peak tensors from the contraction tree.

4.3.3 Computing Target Layouts for Peak Tensors

Once the peak nodes are identified, the strategy must compute a target layout for each peak tensor. That is a desired ordering of its indices that aligns with GEMM-friendly patterns and minimizes the permutation cost to achieve that layout. The target layout for a peak tensor is inferred from its position in the contraction tree and the layouts of its neighboring tensors. The function implements a systematic strategy that considers several cases. Figure 4.7 shows the core pseudocode for this target-layout computation.

Leaf Node Handling

If a peak tensor is a leaf (an original input tensor), its target layout is simply its current input index ordering. This reflects the assumption that initial tensors, regardless of how they are loaded into memory, are presented in a canonical order by the user and that permuting them before contraction is costly.

Root Node Handling

If a peak tensor is the root of the contraction tree (produced as the final result of all contractions), its target layout is determined by sorting its indices by size. This heuristic is based on the observation that GEMM kernels often benefit from contiguous access patterns over larger strides, and larger strides correspond to larger dimension sizes in typical tensor representations [3].

Internal Node Handling with Ancestor Contraction

For internal nodes (non-leaf, non-root tensors), the target layout is derived from the contraction operation that produces the node's tensor and from the desired layouts of its ancestors if they have been previously computed. The logic is as follows:

First, identify the parent contraction node and retrieve its left and right input tensors and the result tensor of that contraction. Then, classify the indices of the peak tensor into two sets: those that participate in the contraction (contracted indices) and those that remain in the output (free indices).

The strategy then queries the a dictionary of already determined layouts to check if the parent node or the sibling node has a precomputed desired layout. If either exists, the free and contracted indices are sorted to respect the corresponding layout. Specifically, free indices are sorted to maintain the relative ordering they have in the parent's desired layout while contracted indices are similarly aligned with the sibling's layout if available, or else sorted by size as a fallback.

The final target layout is constructed by concatenating the sorted free and contracted indices in an order that respects the position of the peak tensor in the binary contraction. If the peak tensor is the left child, the layout is free indices followed by contracted indices. If it is the right child, the order is reversed. This asymmetry is deliberate since it reflects the typical operand roles in a matrix-multiplication operation where the left operand is the A matrix and the right is the B matrix, and different orderings of indices optimize different data-access patterns for each position.

4.3.4 Index Classification and Interleaving Analysis

A critical challenge in the permutation planning phase is determining how to arrange the indices of tensors involved in a contraction so that they satisfy two potentially conflicting objectives: (1) the indices of the two input tensors should be arranged so that their contracted indices are contiguous and aligned, and (2) the free indices should be arranged in an order that matches the desired layout of the parent tensor.

This conflict arises when the desired free-index order interleaves free indices from the left and right children. For example, if the desired layout is $[l_1, r_1, l_2, r_2]$ where l_i are left free indices and r_i are right free indices, then it is impossible to realize this layout by arranging the left tensor as $[l_1, l_2, \ldots, c_1, c_2, \ldots]$ and the right tensor as $[c_1, c_2, \ldots, r_1, r_2, \ldots]$ without permuting the contracted indices or introducing sliced indices. The strategy checks whether the desired order follows the pattern L^*R^*, i.e., a prefix of left-free indices followed by a suffix of right-free indices, or any contiguous subsequence thereof. An iteration through the desired free indices in order takes place where a flag is maintained that tracks whether a right-free index has been encountered. If a left-free index is encountered after setting the flag to true, the pattern is broken and the function returns false. Otherwise, it returns true, indicating that the desired pattern is achievable via a straightforward concatenation without slicing. Figure 4.8 shows the check performed by the algorithm.

Selecting Sliced Indices for Interleaved Patterns

When the desired free-index order is interleaved and does not follow the L^*R^* pattern, the strategy falls back to a slicing approach. The idea is to partition the free indices into two sets: those that will be kept in their natural left-then-right order (the "representable" portion) and those that will be marked as sliced (the "irregular" portion). Sliced indices are treated as participating in slicing operations rather than in the main permutation, and their handling is deferred to a lower level of the execution stack.

Finding the partition that maximizes the number of representable indices is done by computing, for each possible cutoff point, the length of the longest subsequence that follows the L^*R^* pattern. The function precomputes prefix counts of left indices and suffix counts of right indices, then iterates over all possible switch points to find the one that maximizes the combined count, in essence a classic dynamic programming problem. The pseudocode for this partitioning is shown in Figure 4.9.

Once the best partition is found, all indices not included in the representable portion are added to the sliced set. These sliced indices are then removed from consideration in the subsequent layout computation, effectively reducing the problem to a smaller subset of indices that can be arranged without interleaving.

4.3.5 Recursive Layout Planning

With peak tensors identified and their target layouts computed, the strategy must propagate compatible layouts throughout the contraction tree, ensuring that all non-peak tensors receive permutations that are compatible with their parent's expectations.

This propagation starts from the root of the contraction tree and recursively descends to both children. At each internal node, the optimal layouts are computed for the left and right input tensors based on the node's position in the tree and the requirements of its siblings. The recursive planning logic is summarized in Figure 4.10.

Base Case and Termination

The planner first checks whether the current node represents a valid internal contraction step. If any of the left child, right child, or contraction step is missing or null, the planning returns immediately without further processing. This handles leaf nodes and degenerate cases where contraction information is incomplete.

Extracting Contraction Structure

For valid internal nodes, the planner retrieves the left input tensor, right input tensor, and result tensor. It then partitions the indices of these tensors into contracted and free sets. Contracted indices are those that appear in both input tensors and are eliminated by the contraction. Free indices are those that remain in the output. The planner next determines the desired order for free indices. If the current node's desired layout has been precomputed (e.g., because the node is in the peak set or because a parent has already specified requirements), the planner extracts the free indices from that layout, preserving their relative order. Otherwise, it computes a default order by sorting free indices by size: first the left-free indices sorted by size, then the right-free indices sorted by size.

Index Partitioning for Slicing

The planner determines which free indices (if any) should be treated as sliced. This partitioning is based on whether the desired free-index order follows the L^*R^* pattern. If it does, no slicing is needed. Otherwise, the minimal set of indices required to break the interleaving pattern is identified and marked as sliced.

Constructing Target Layouts

With free and contracted indices partitioned into sliced and representable subsets, the planner constructs three target layouts: one for the left input, one for the right input,

and one for the result.

For the left input, the layout is: (1) sliced indices from the left, sorted by size, (2) representable free indices from the left, sorted by their position in the desired free order, (3) contracted indices, sorted by size. This ordering is motivated by the observation that sliced indices should appear first to group them contiguously, followed by the representable free indices in the desired order, and finally contracted indices at the end where they can be aligned with the right input.

For the right input, the layout is: (1) contracted indices, sorted by size, (2) sliced indices from the right, sorted by size, (3) representable free indices from the right, sorted by their position in the desired free order. The contracted indices appear first because they must align with the left input's contracted indices for efficient computation. The sliced indices follow, and finally the right-free indices, which will become output free indices.

For the result tensor, the layout is a concatenation of all sliced indices (first the left-sliced, then the right-sliced, both sorted by size), followed by all representable free indices (left representable sorted by desired order, then right representable sorted by desired order). This layout ensures that the output tensor's free indices appear in the desired order while sliced indices are grouped at the beginning.

Conditional Layout Assignment and Recursion

Before assigning these target layouts to the input tensors, children nodes need to be checked incase they're frozen (marked as a peak). If a node is not frozen, its layout gets assigned. Otherwise, the assignment is skipped, preserving the node's precomputed layout. Similarly, if the current node itself is not frozen, its computed result layout is assigned to the result tensor. If it is frozen, the assignment is skipped.

After layout assignment, this process is recursively called on both child nodes, continuing the tree traversal and propagating layouts downward through the tree.

The order of operations (assigning layouts to children before recursing, and recursing to both children before returning) ensures that layouts are computed in a top-down manner. This means parent layouts inform child layouts, and all siblings within a subtree respect the constraints imposed by their parent.

4.3.6 Layout Assignment and Permutation Computation

Once the recursive planning phase has computed a target layout for each tensor in the contraction tree, the strategy must assign these layouts and compute the corresponding permutation operations. The assignment def is shown in Figure 4.11.

With each tensor in hand, a permutation vector is computed that transforms the tensor's current index ordering to the target layout. Formally, a permutation vector π is found such that $\pi[i]$ indicates the position in the current index list of the index that should appear at position i in the target layout.

4.3.7 Algorithmic Complexity

The time complexity of the greedy strategy can be analyzed as follows. Let n denote the number of initial tensors, p denote the length of the contraction path, d denote the average tensor rank (number of indices), and r denote the number of peak tensors ($r = \min(k, n + p)$).

The peak node identification phase involves computing memory costs for all candidate tensors and sorting them. This is $O((n + p) \log(n + p))$.

The target layout computation phase finds the target peak layout for each of the r peak nodes. Each call performs constant-time or $O(d \log d)$ operations (sorting indices). This phase is thus $O(r \cdot d \log d)$.

The recursive planning phase traverses the entire contraction tree (which has $O(n + p)$ nodes) and performs $O(d)$ work per node (sorting indices, partitioning sets). This phase is $O((n + p) \cdot d)$.

The permutation computation phase computes $O(n + p)$ permutations, each requiring $O(d^2)$ work in the worst case (building a permutation vector for a rank-d tensor). This phase is $O((n + p) \cdot d^2)$.

Overall, the algorithm's complexity is $O((n + p) \cdot d^2 + (n + p) \log(n + p))$, which is dominated by the permutation computation phase. For small ranks d (typical in practical tensor networks) and moderate path lengths p, this is effectively linear in the problem size.

4.3.8 Design Rationales

The greedy strategy makes several deliberate design choices that reflect a balance between generality, efficiency, and ease of implementation.

Greedy vs. Optimal

The strategy is intentionally greedy and not optimal. Finding the globally optimal permutation plan would require solving a constraint satisfaction or optimization problem over the full space of permutations, which is computationally intractable. The greedy approach sacrifices optimality for computational tractability, with the assumption that local optimization around peak tensors yields sufficiently good results in practice.

Persistent Path Preparation

The construction of the `PersistentContractionPath` upfront adds a one-time cost but amortizes the cost of subsequent queries. Without this preparation, every query to an intermediate network state would require replaying the entire contraction sequence, making the algorithm's complexity quadratic or worse. By investing in persistence, the algorithm trades space for time, a worthwhile trade-off for typical problem sizes.

Slicing as a Fallback

The use of slicing to handle interleaved index patterns is a design choice that defers the complexity of index reordering to the lower-level execution layer (the TCCG generator and HPTT primitives). This keeps the permutation strategy implementation relatively simple while allowing the lower layers to employ optimized techniques for slicing when necessary.

4.4 The Full Picture

As figure 4.12 illustrates, the completed pipeline has three broad segments: input preparation, permutation optimization, and native kernel generation. The user begins by specifying a tensor network and intended contraction path. The network is represented in the `TensorNetwork` abstraction, which centralizes tensor metadata, index sets, and dimension sizes.

The first stage of the pipeline transforms these inputs into an internal, persistent form. The `PersistentContractionPath` preserves every intermediate state of the contraction sequence, allowing the optimizer to analyze both original tensors and temporary results without recomputing earlier steps.

The greedy algorithm then analyzes the persistent state. It identifies the largest tensors in memory as the critical "peaks", computes GEMM-friendly target layouts for them, and propagates compatible layouts across the contraction tree.

Once the optimizer has produced a sequence of tensor operations, the pipeline compiles them into an execution-ready form. A TCCG description is generated for each contracted tensor pair, discovers the generated function signature, emits a `pybind11` wrapper, and invokes the native C++ compiler. The generated code is linked against HPTT and the low-level tensor compute kernels needed for transpose and packing.

At runtime, the compiled module is dynamically loaded and executed. The tensor operations produced by the optimizer are mapped to concrete layout transformations and contraction kernels, and the final tensor result is produced by composing the permuted inputs, contracted intermediates, and the generated native kernels.

```
def COMPUTE_PEAK_TARGET_LAYOUT(node, desired_layouts):
    if node.is_leaf:
        return node.input_indices

    if node.is_root:
        return sort_by_size(node.input_indices)

    parent = node.parent
    left, right, _ = get_step_tensors(parent.contraction_step)
    contracted = get_contracted_indices(node.tensor, sibling(node))
    free = node.input_indices - contracted

    if desired_layouts.contains(parent.id):
        free_order = sort_by_layout(free, desired_layouts[parent.id])
    else:
        free_order = sort_by_size(free)

    if desired_layouts.contains(sibling(node).id):
        contracted_order = sort_by_layout(contracted,
            desired_layouts[sibling(node).id])
    else:
        contracted_order = sort_by_size(contracted)

    if node is left child:
        return concatenate(free_order, contracted_order)
    else:
        return concatenate(contracted_order, free_order)
```

Figure 4.7: Pseudocode for computing the target layout of a peak tensor.

```
def HAS_LEFT_THEN_RIGHT_ORDER(desired_free, left_free, right_free):
    seen_right = false
    for index in desired_free:
        if index in right_free:
            seen_right = true
        else if index in left_free and seen_right:
            return false
    return true
```

Figure 4.8: Pseudocode for checking whether the free-index order is representable without slicing.

```
def SELECT_SLICED_INDICES(desired_free, left_free, right_free):
    best_keep = -1
    best_split = 0
    prefix_left = compute_prefix_counts(desired_free, left_free)
    suffix_right = compute_suffix_counts(desired_free, right_free)

    for split from 0 to length(desired_free):
        keep = prefix_left[split] + suffix_right[split]
        if keep > best_keep:
            best_keep = keep
            best_split = split

    sliced = {}
    for i, index in enumerate(desired_free):
        keep_left = (i < best_split and index in left_free)
        keep_right = (i >= best_split and index in right_free)
        if not (keep_left or keep_right):
            sliced.add(index)

    return sliced
```

Figure 4.9: Pseudocode for selecting sliced indices when free-index order is interleaved.

```
def PLAN_NODE(node, frozen_nodes, desired_layouts):
    if node is null or node.step is null:
        return

    left, right, result = get_step_tensors(node.step)
    contracted = get_contracted_indices(left, right)
    left_free = left.indices - contracted
    right_free = right.indices - contracted

    desired_free = determine_desired_free_order(node, desired_layouts)
    if HAS_LEFT_THEN_RIGHT_ORDER(desired_free, left_free, right_free):
        sliced = {}
    else:
        sliced = SELECT_SLICED_INDICES(desired_free, left_free, right_free)

    left_target = build_left_target(left_free,
        sliced, contracted, desired_free)
    right_target = build_right_target(right_free,
        sliced, contracted, desired_free)
    result_target = build_result_target(left_free,
        right_free, sliced, desired_free)

    if node.left not in frozen_nodes:
        assign_layout(node.left, left_target)
    if node.right not in frozen_nodes:
        assign_layout(node.right, right_target)
    if node not in frozen_nodes:
        assign_layout(node, result_target)

    PLAN_NODE(node.left, frozen_nodes, desired_layouts)
    PLAN_NODE(node.right, frozen_nodes, desired_layouts)
```

Figure 4.10: Pseudocode for the recursive layout planning phase.

```
def ASSIGN_PERMUTATION(node, target_layout):
    tensor = get_node_tensor(node)
    permutation = compute_permutation(tensor.indices, target_layout)

    if node.is_leaf:
        initial_permutations[node.position] = permutation
    else:
        intermediate_permutations[node.step] = permutation

    desired_layouts[node.id] = target_layout
```

Figure 4.11: Pseudocode for assigning a permutation to a tensor node after layout planning.

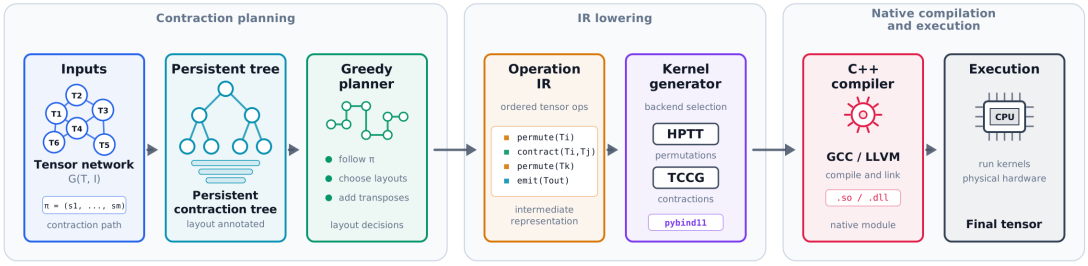


Figure 4.12: End-to-end pipeline from user tensor network to final compiled execution result.

5 Evaluation

This chapter evaluates the proposed permutation-planning and execution pipeline. We describe the experimental methodology, detail the hardware and software configuration used for the measurements, and present quantitative results for runtime, peak memory usage, and memory movement that compare the baseline and optimized strategies.

5.1 Setup

All experiments reported in this chapter were executed on a single central machine dedicated to benchmarking and performance validation. The machine is built around a 12th-generation Intel Core i5 processor in specific the 12th Gen Intel(R) Core(TM) i5-12450H, and was configured to expose the AVX2 instruction set. The system used 16 GB of main memory for every run reported here. Kernel generation and compilation were performed using the GNU compiler version 11.4.0, and the specific TCCG instance as pf v0.1.1. Experiments targeted double precision arithmetic and relied on a conservative generator setting of `maxImplementations = 1` to ensure stable and comparable kernels across runs.

Parallel execution was driven by the GNU OpenMP runtime and controlled through the environment affinity hints stored alongside the generator configuration `GOMP_CPU_AFFINITY=0-12`. On this machine the measured core topology presented eight logical cores and twelve virtual cores; accordingly the affinity and thread settings were explicitly adjusted to make the runtime parallelism unambiguous. To validate portability and to check for operating-system-level effects, the same experiments were also executed on Windows Subsystem for Linux 2 (WSL2) running Ubuntu 22.04.

5.2 Impact of ignoring the largest intermediate tensor

The first empirical study was designed to evaluate the central hypothesis of this work that intentionally avoiding permutations for the largest intermediate tensors in a contraction sequence can reduce the overall peak and intermediate memory requirements without compromising the local contraction choices. To test this hypothesis at scale we generated a corpus of 5,200 random tensor networks with between seven and twenty

tensors each and compared two execution strategies. The first strategy is a conservative baseline that always applies the locally optimal permutation for each contraction. The second strategy freezes the single largest intermediate tensor ($k = 1$) and therefore does not permute it prior to contractions. For both strategies we measured the peak resident memory and the summed intermediate memory movement produced during the complete contraction.

Figure 5.1 displays the aggregated results. The figure summarises the relative difference between the baseline and the ignore-largest strategy. Across the whole corpus the ignore-largest strategy produced a substantial reduction in measured memory. On average the peak and intermediate memory usage was lower by approximately 45% when the largest intermediate tensor was left unpermuted. Moreover, the benefit is not uniform across network sizes. As network size increases (more tensors per network) the effect becomes more pronounced, reflecting the intuition that larger contraction trees increasingly channel data movement through a small set of large intermediates that act as choke points. In these cases avoiding permutations on the dominant intermediate relieves the most significant memory-traffic hotspot and therefore yields larger savings.

These results support the claim that treating layout for the largest intermediate tensors as a first-class design choice can deliver practical memory reductions in realistic contraction workloads. The remainder of this chapter examines representative individual networks from the corpus, quantifies variance and outliers, and investigates how the observed savings depend on rank distributions and contraction topologies.

5.3 Theoretical Analysis

To evaluate the algorithm in a setting where random tensor allocations and permutation costs do not introduce variance, we performed a pure theoretical analysis of the greedy permutation strategy against established baselines. The central question was whether the empirical savings observed in the initial experiment (approximately 45% on average for the top-1 frozen tensor) would be consistent across a wider range of tensor network topologies and sizes. To answer this, we generated contraction paths using the state-of-the-art library `cotengra` for a large corpus of synthetic tensor networks, then applied our greedy permutation strategy (with varying values of k, the number of frozen tensors) to the same paths and computed the theoretical peak memory consumption that would result.

The test corpus comprised 18,180 distinct tensor networks with topologies ranging from three to seven tensors. Each network was randomly generated with independent choices for tensor rank, input dimension sizes, contraction structure, and output index sets. This diversity ensures that the theoretical analysis covers a representative space

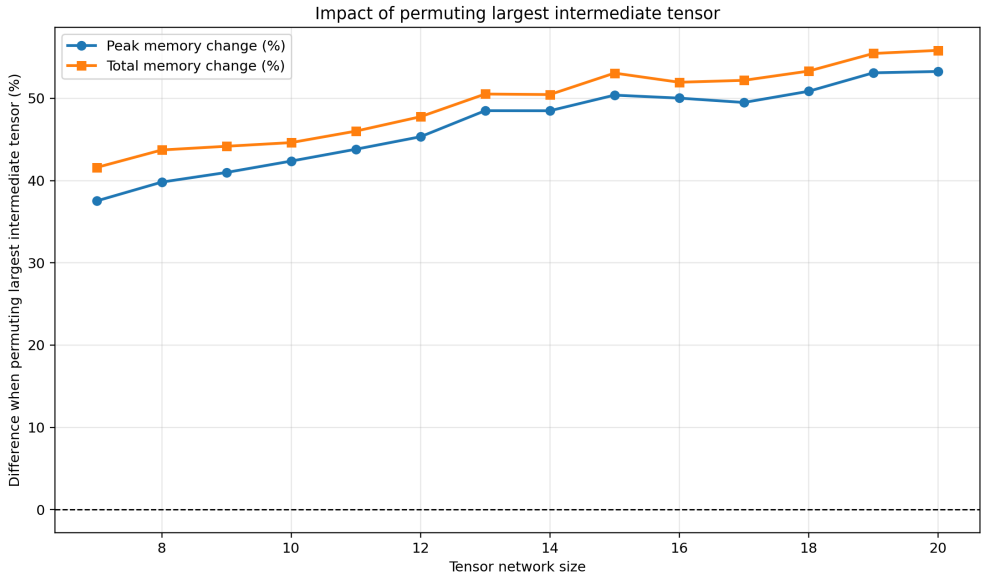


Figure 5.1: Impact of ignoring permutation of the largest intermediate tensor on peak and intermediate memory.

of practical tensor contraction problems. Because the theoretical analysis tests only the permutation and layout choices and not the actual memory movement of large numerical arrays, we could evaluate this large corpus efficiently without incurring the cost of full experimental runs. Regardless, the decision to limit network sizes to the 3–7 tensor range was still made deliberately since the same sample set will be used to validate the non-theoretical memory movement advantages and disadvantages of the greedy strategy in subsequent sections.

Figure 5.2 shows the absolute theoretical peak memory for each of the 18,180 networks, grouped by the sizes of the tensor netwokrs. The vertical axis represents the theoretical peak (in MBs). As k increases, more tensors are frozen, the bars shifts downward, reflecting a consistent reduction in the predicted memory footprint. The bars with lower values indicate that the greedy strategy with even modest k values (e.g., $k = 1$ or $k = 2$) produces theoretical savings. Notably the savings align with the initial experiment's finding. Freezing only the single largest intermediate yields theoretical improvements consistent with the 45% reduction observed empirically, validating the hypothesis that the algorithm's layout decisions transfer predictably to real executions. Finally, it is important to note that the baseline, although independent of k, was ran multiple times to ensure data consistency.

Figure 5.3 quantifies these savings as percentage improvements over the baseline.

The bar chart shows the average and distribution of relative improvement for each value of k. Across the corpus, freezing the top-1 tensor yields an average theoretical peak memory reduction of approximately 30%; increasing to top-3 frozen tensors boosts the average improvement to around 42%. These numbers indicate that even conservative values of k yield significant theoretical benefits.

The consistency between the theoretical predictions (Figures 5.2 and 5.3) and the empirical observations from the initial experiment (Figure 5.1) provides confidence that the greedy algorithm's layout decisions are well-founded and that its benefits are robust across varying network sizes and topologies. The remainder of this chapter presents actual memory movement data for selected networks from the 3–7 tensor range, to attempt validate that the theoretical savings translate to practical performance gains on the central machine described in Section 5.1.

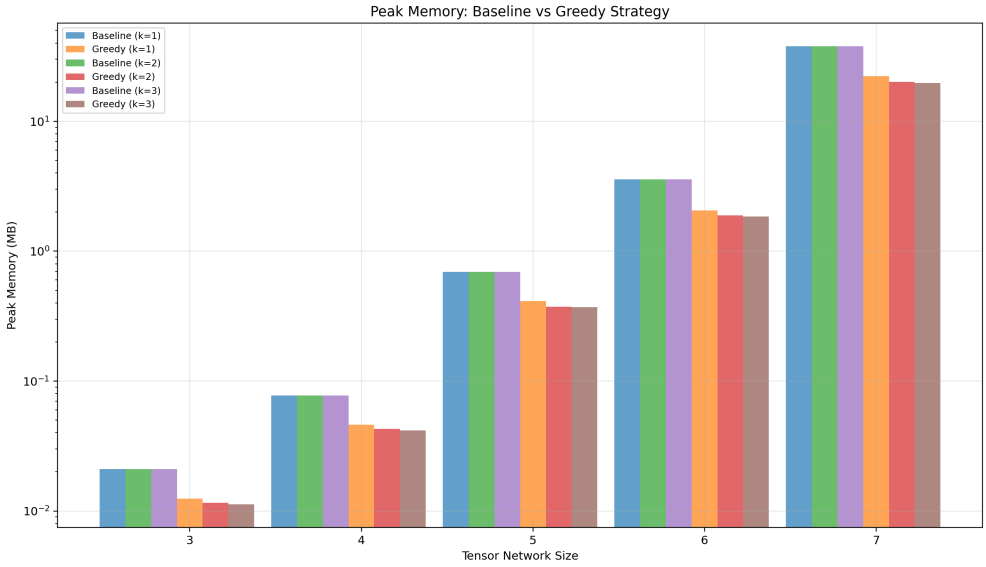


Figure 5.2: Absolute theoretical peak memory of the baseline (`cotengra`) against the proposed greedy algorithm.

5.4 Practical Analysis

After establishing the theoretical memory behaviour of the greedy permutation strategy, we performed a practical analysis to determine whether these theoretical savings translate into measurable improvements during actual execution. This distinction is important because the theoretical model only accounts for tensor sizes, intermediate

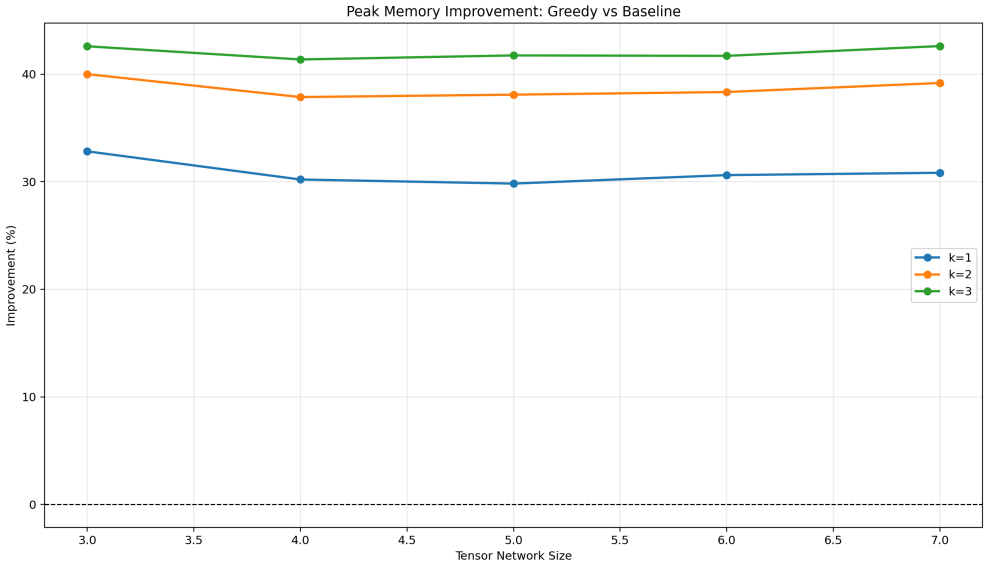


Figure 5.3: Percentage improvement in theoretical peak memory from using the greedy strategy over the baseline strategy (`cotengra`).

tensors, and planned memory movement. In contrast, a practical execution measures the full behaviour of the running program, including allocator overhead, runtime buffers, Python and library metadata, cached allocations, temporary arrays, and any memory retained by the operating system. Therefore, the practical results should not be expected to match the theoretical estimates exactly.

Figure 5.4 shows the measured peak memory for the state-of-the-art baseline and greedy strategies across tensor network sizes from three to seven tensors and for different values of k. Unlike the theoretical results, where the greedy strategy showed clear and consistent reductions in peak memory, the practical measurements show only minor differences between the two strategies. In most cases, the measured peak memory of the greedy strategy is almost identical to the baseline. The observed improvements are therefore minimal, generally remaining within a very small range. For peak memory specifically, the improvement is close to zero for smaller networks and becomes slightly negative for the largest tested networks, as shown in Figure 5.5. This indicates that, in the practical implementation, the greedy strategy does not consistently reduce the resident memory footprint despite its theoretical advantage.

A major reason for this gap is the high level of optimization already present in the state-of-the-art baseline library, `cotengra`. Although our algorithm changes the permutation decisions and attempts to reduce memory movement from a layout-planning

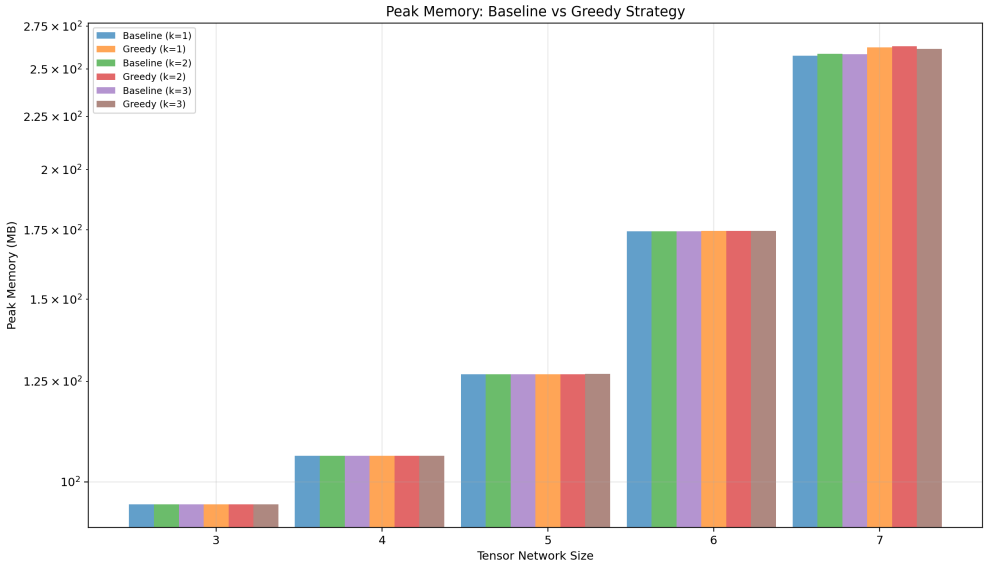


Figure 5.4: Measured peak memory of the baseline strategy against the proposed greedy strategy for different tensor network sizes and values of k.

perspective, `cotengra` is already heavily optimized at the execution level. In particular, it attempts to minimize memory footprint during contraction by reusing existing storage whenever possible, relying on views where applicable, and avoiding unnecessary materialization of intermediate tensors. By contrast, the current implementation of the proposed algorithm often allocates new tensors during contraction and permutation steps. These additional allocations reduce the practical benefit of the theoretical memory savings, because memory saved at the layout-planning level can be offset by memory introduced at the execution level.

Another important observation is that state-of-the-art tensor contraction libraries often benefit from a tightly coupled optimization stack. In such systems, contraction path search, tensor contraction execution, layout handling, and low-level compilation are optimized together. This coupling allows the implementation to make decisions that are globally efficient across several layers of the execution pipeline. However, it also makes these systems less modular, since a new contraction or permutation algorithm cannot easily reuse only the low-level optimization layer without adapting to the entire stack. In contrast, the proposed algorithm focuses mainly on contraction-layout optimization after the contraction path has been selected. It does not yet include the same degree of integration with low-level compilation, allocator-aware execution, or memory-pool management. This explains why the practical gains are smaller than the

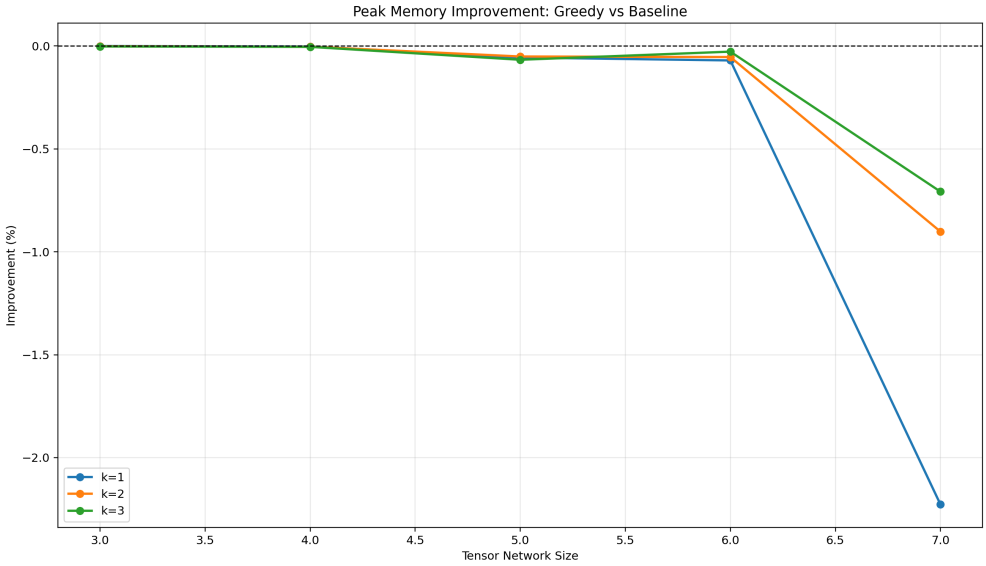


Figure 5.5: Measured peak memory improvement of the greedy strategy over the baseline strategy.

theoretical gains.

Figure 5.6 further illustrates this point by showing the total memory improvement of the greedy strategy over the baseline. For most smaller networks, the total memory difference remains close to zero, with small positive improvements in some cases. However, the larger networks show that practical execution can also introduce regressions, especially when the cost of additional allocations and temporary tensors dominates the benefit of avoiding selected permutations. This result does not invalidate the theoretical analysis. Rather, it shows that theoretical memory reduction is necessary but not sufficient for practical memory reduction. To achieve practical gains, the layout algorithm must be integrated with an execution system that can utilize those savings all the way down to allocation, contraction, and kernel execution.

To further evaluate whether the greedy permutation strategy produces measurable memory savings in practice, we repeated the practical experiment on the same dataset used in the previous subsection. In contrast to the earlier comparison, where the observed gains were minimal, this experiment compares the greedy strategy against a simpler baseline strategy that does not attempt to preserve the layout of large intermediate tensors. This baseline therefore represents a more direct reference point for measuring the practical benefit of the proposed memory-aware layout planning since it builds on top of the proposed stack.

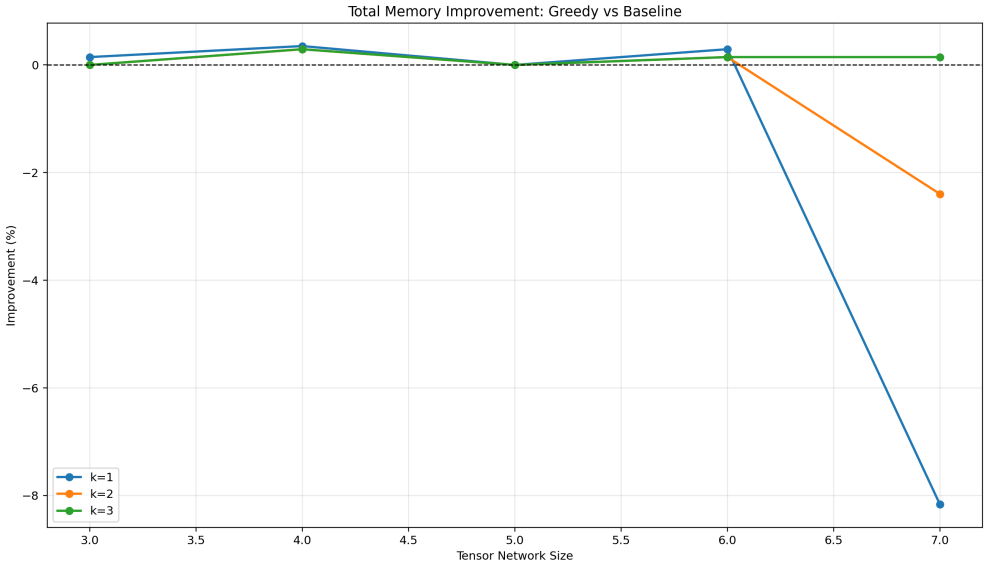


Figure 5.6: Measured total memory improvement of the greedy strategy over the baseline strategy.

Figure 5.7 shows the measured peak memory usage for tensor networks of size three to seven. For each network size, the baseline memory remains almost unchanged across the different values of k, while the greedy strategy consistently reduces the measured peak memory. The absolute difference becomes more visible as the tensor network size increases. This behavior is also reflected in Figure 5.8, which reports the relative improvement of the greedy strategy over the baseline. Although the improvement is small for networks of size three and four, it grows steadily for larger networks, reaching its largest values for networks of size seven.

However, the practical improvements remain substantially smaller than the 30–40% reductions predicted by the theoretical memory model. This discrepancy is expected since as mentioned before the theoretical analysis isolates the memory contribution of tensor intermediates and permutation buffers, whereas the practical measurement records the peak memory of the entire program. As a result, the measured peak includes additional sources of memory consumption such as the Python runtime, library overhead, metadata, allocator behaviour, generated kernels, and other temporary buffers that are not directly affected by the greedy tensor-layout strategy. These fixed and implementation-dependent costs dilute the relative improvement that can be observed at the process level.

It is also important to note that the measured practical peak memory is generally

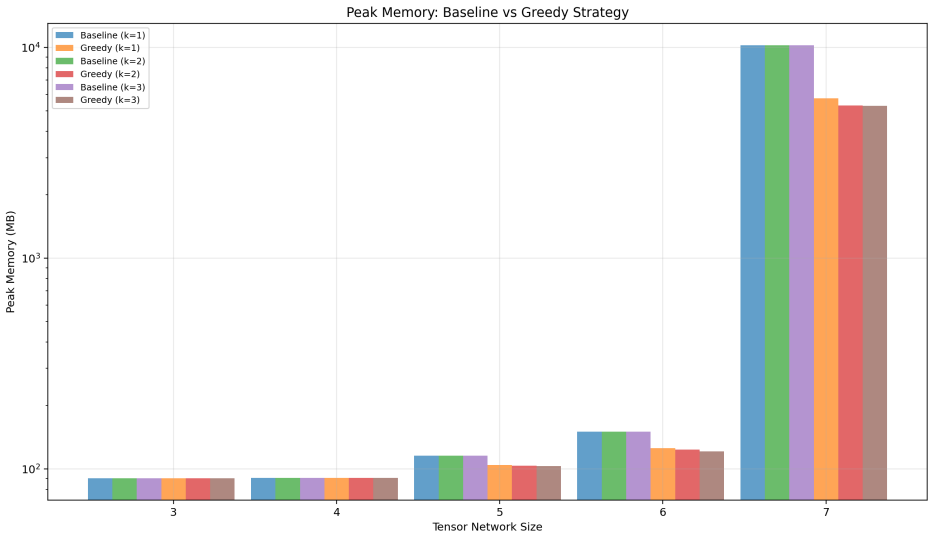


Figure 5.7: Measured peak memory usage of the naive baseline and greedy strategy for different tensor network sizes and values of k.

larger than the theoretical peak memory. This is expected because the practical measurement uses the resident set size (RSS) of the process. RSS captures the memory currently resident in physical memory for the whole program as well. Therefore, the practical results should be interpreted as an end-to-end system measurement, while the theoretical results should be interpreted as an idealized estimate of tensor-level memory behaviour.

Despite the limited practical improvement, the results remain encouraging. The proposed algorithm and library do not yet include the full set of coupled optimizations used by mature tensor contraction frameworks, yet they still perform identical to the state-of-the-art baseline in most practical cases and even outperform upon existing naive implementations. This is significant because the baseline benefits from years of coupled optimization across path finding, contraction execution, memory handling, and low-level implementation. With stronger integration into the execution layer, especially through memory reuse and compiler-level optimization, the theoretical advantages of the algorithm are likely to translate more directly into practical gains.

5.5 Future Work

The practical analysis highlights a broader limitation in the current tensor contraction ecosystem. Many state-of-the-art systems achieve strong performance by tightly

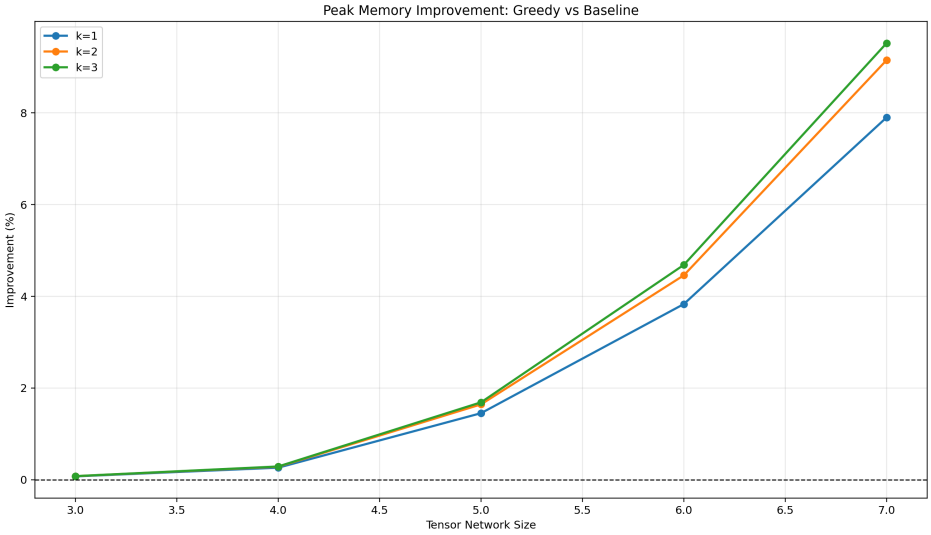


Figure 5.8: Relative peak memory improvement of the greedy strategy over the naive baseline strategy.

coupling contraction path optimization, layout decisions, contraction execution, and low-level compilation. While this produces highly optimized implementations, it also makes it difficult for new algorithms to reuse only the lower-level performance infrastructure. As a result, researchers proposing new path-selection or permutation-optimization algorithms may need to reimplement large parts of the execution stack before their ideas can be evaluated fairly in practice, as with our attempt here.

A promising direction for future work is the development of a tensor network compiler. Such a compiler would separate high-level contraction optimization from low-level code generation and memory management. In this model, algorithms such as the proposed greedy permutation strategy would output an intermediate representation describing the contraction path, tensor layouts, frozen tensors, permutation constraints, and memory-reuse opportunities. The compiler would then lower this representation into optimized kernels, memory plans, and execution schedules. This would allow new algorithms to benefit from the same low-level optimizations currently available in mature libraries without requiring each algorithm to reimplement the complete stack.

This separation would also make practical evaluation more reliable. If different algorithms target the same compiler backend, their differences can be measured at the algorithmic level rather than being dominated by differences in implementation quality. Additionally, a compiler-based approach could introduce advanced features such as allocator-aware contraction scheduling, automatic view reuse, memory-pool

planning, fusion of permutation and contraction operations, and hardware-specific kernel generation. These optimizations are difficult to implement independently inside every new algorithm, but they are natural responsibilities for a shared compiler infrastructure.

Therefore, the main future direction suggested by this work is not only to improve the greedy strategy itself, but to improve the surrounding ecosystem in which such strategies are executed. A modular tensor network compiler would make the field more extensible, more comparable, and more practically useful. It would allow researchers to focus on new contraction and layout ideas while still benefiting from state-of-the-art low-level optimization. In turn, this could help close the gap between theoretical memory savings and practical end-to-end performance.

6 Conclusion

This thesis investigated memory-aware tensor network contraction, focusing on reducing unnecessary memory movement through permutation-aware contraction planning. The proposed greedy strategy showed promising theoretical potential by avoiding costly layout transformations, especially for large intermediate tensors. However, the practical evaluation showed only minimal improvements, with results generally remaining close to the baseline and occasionally becoming slightly negative. This gap is mainly due to the strong optimizations already present in state-of-the-art libraries such as `cotengra`, which combine contraction path search, memory reuse, view-based execution, and low-level implementation optimizations.

Despite the limited practical gains, the results provide a strong foundation for future research. The proposed algorithm performed close to the state-of-the-art baseline without benefiting from the same level of tightly coupled low-level optimization or execution stack. This suggests that the core idea remains valuable, but future work should focus on preserving theoretical savings through the full execution pipeline. A promising direction is the development of a modular tensor network compiler that separates high-level contraction and layout optimization from low-level memory management and kernel generation, allowing new algorithms to benefit from optimized backends without reimplementing the entire stack.

Abbreviations

TNC Tensor network contraction

FLOP floating-point operations

HPC high-performance computing

BLAS Basic Linear Algebra Subprograms

MPS Matrix Product State

GEMM general matrix-matrix multiplication

TCCG Tensor Contraction Code Generator

HPTT High-Performance Tensor Transposition

List of Figures

Bibliography

[1] F. Arute, K. Arya, R. Babbush, D. Bacon, J. C. Bardin, R. Barends, R. Biswas, S. Boixo, F. G. Brandao, D. A. Buell, et al. "Quantum supremacy using a programmable superconducting processor." In: *nature* 574.7779 (2019), pp. 505–510.

[2] J. Biamonte and V. Bergholm. "Tensor networks in a nutshell." In: *arXiv preprint arXiv:1708.00006* (2017).

[3] M. O. Blom, K. F. Rietveld, and R. V. van Nieuwpoort. "Multi-Strided Access Patterns to Boost Hardware Prefetching." In: *Proceedings of the 16th ACM/SPEC International Conference on Performance Engineering*. 2025, pp. 204–215.

[4] J. Brandejs, N. Hörnblad, E. F. Valeev, A. Heinecke, J. Hammond, D. Matthews, and P. Bientinesi. "Tensor Algebra Processing Primitives (TAPP): Towards a Standard for Tensor Operations." In: *arXiv preprint arXiv:2601.07827* (2026).

[5] E. F. Dumitrescu, A. L. Fisher, T. D. Goodrich, T. S. Humble, B. D. Sullivan, and A. L. Wright. "Benchmarking treewidth as a practical component of tensor network simulations." In: *PloS one* 13.12 (2018), e0207827.

[6] G. Evenbly and R. N. Pfeifer. "Improving the efficiency of variational tensor network algorithms." In: *Physical Review B* 89.24 (2014), p. 245118.

[7] M. Frigo and S. G. Johnson. "The Design and Implementation of FFTW3." In: *Proceedings of the IEEE* 93.2 (2005), pp. 216–231. DOI: 10.1109/JPROC.2004.840301.

[8] M. D. García and A. Márquez Romero. "Survey on Computational Applications of Tensor-Network Simulations." In: *IEEE Access* 12 (2024), pp. 193212–193228. DOI: 10.1109/ACCESS.2024.3519676.

[9] V. Gogate and R. Dechter. "A complete anytime algorithm for treewidth." In: *arXiv preprint arXiv:1207.4109* (2012).

[10] K. Goto and R. A. v. d. Geijn. "Anatomy of high-performance matrix multiplication." In: *ACM Transactions on Mathematical Software (TOMS)* 34.3 (2008), pp. 1–25.

[11] J. Gray and S. Kourtis. "Hyper-optimized tensor network contraction." In: *Quantum* 5 (2021), p. 410.

[12] M. Hamann and B. Strasser. "Graph bisection with pareto optimization." In: *Journal of Experimental Algorithmics (JEA)* 23 (2018), pp. 1–34.

[13] S. Hirata. "Tensor contraction engine: Abstraction and automated parallel implementation of configuration-interaction, coupled-cluster, and many-body perturbation theories." In: *The Journal of Physical Chemistry A* 107.46 (2003), pp. 9887–9897.

[14] C. Huang, F. Zhang, M. Newman, X. Ni, D. Ding, J. Cai, X. Gao, T. Wang, F. Wu, G. Zhang, et al. "Efficient parallelization of tensor network contraction for simulating quantum computation." In: *Nature Computational Science* 1.9 (2021), pp. 578–587.

[15] C. L. Lawson, R. J. Hanson, D. R. Kincaid, and F. T. Krogh. "Basic linear algebra subprograms for Fortran usage." In: *ACM Transactions on Mathematical Software (TOMS)* 5.3 (1979), pp. 308–323.

[16] R. Li, A. Sukumaran-Rajam, R. Veras, T. M. Low, F. Rastello, A. Rountev, and P. Sadayappan. "Analytical cache modeling and tilesize optimization for tensor contractions." In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 2019, pp. 1–13.

[17] I. L. Markov and Y. Shi. "Simulating quantum computation by contracting tensor networks." In: *SIAM Journal on Computing* 38.3 (2008), pp. 963–981.

[18] I. V. Oseledets. "Tensor-train decomposition." In: *SIAM Journal on Scientific Computing* 33.5 (2011), pp. 2295–2317.

[19] N. Park, B. Hong, and V. K. Prasanna. "Tiling, block data layout, and memory hierarchy performance." In: *IEEE Transactions on Parallel and Distributed Systems* 14.7 (2003), pp. 640–654.

[20] R. N. Pfeifer, J. Haegeman, and F. Verstraete. "Faster identification of optimal contraction sequences for tensor networks." In: *Physical Review E* 90.3 (2014), p. 033315.

[21] F. Schindler and A. S. Jermyn. "Algorithms for tensor network contraction ordering." In: *Machine Learning: Science and Technology* 1.3 (2020), p. 035001.

[22] U. Schollwöck. "The density-matrix renormalization group: a short introduction." In: *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 369.1946 (2011), pp. 2643–2661.

[23] P. Springer and P. Bientinesi. "Design of a high-performance GEMM-like Tensor-Tensor Multiplication." In: *CoRR* (2016). arXiv: `1607.00145 [quant-ph]`.

[24] P. Springer and P. Bientinesi. "Design of a high-performance GEMM-like Tensor-Tensor Multiplication." In: *CoRR* abs/1607.00145 (2016). arXiv: `1607.00145`.

[25] P. Springer, T. Su, and P. Bientinesi. "HPTT: a high-performance tensor transposition C++ library." In: June 2017, pp. 56–62. DOI: 10.1145/3091966.3091968.